
Reinforcement Learning – Q-Learning - Tabular & Deep

Coen Büller Johan Baars Mario Magalini

Abstract

We investigate Deep Q-Learning (DQN) as a function approximation method for reinforcement learning in continuous state spaces. We evaluate DQN on the CartPole environment through a hyperparameter ablation study and analyze the impact of stabilization techniques such as Experience Replay and Target Networks. The results show that learning rate, exploration strategy, and data-to-update ratio significantly affect performance, while network architecture has limited influence. Furthermore, stabilization techniques do not provide clear benefits in this setting. Overall, DQN proves to be an effective approach for problems where tabular methods are infeasible.

1. Introduction

In Reinforcement Learning (RL), an agent tries to learn the action-value function $Q(s, a)$ to maximize the cumulative reward over time. In tabular Q-learning, this function is represented explicitly as a table, where each state-action pair (s, a) is assigned a value corresponding to its expected return (Plaats, 2022). However, this is only feasible for small, discrete state spaces. In continuous-state environments, there are infinitely many states, making a tabular representation impractical. To address this limitation, function approximators can be used to represent $Q(s, a)$. Deep Q-Learning (DQN) (Mnih, 2013; Mnih et al., 2015) uses a neural network to approximate the state-action value function $Q(s, a)$, achieving human-level performance when trained on Atari games. In this report, we present the theoretical foundations of DQN and its implementation on the CartPole environment (Barto et al., 1983; Towers et al., 2024). Furthermore, we investigate the stability of DQN under different configurations, specifically analyzing the impact of experience replay and target networks across a range of hyperparameter settings.

2. Theory

From Tabular Q-Learning to Deep Q-Learning Tabular Q-learning represents $Q(s, a)$ explicitly as a table, which becomes impractical when the state-action space $|\mathcal{S} \times \mathcal{A}|$ is large. Deep Q-Learning (DQN) (Mnih, 2013; Mnih et al.,

2015) addresses this limitation by parameterizing the action-value function with a neural network, denoted as $Q_\theta(s, a)$, where θ are trainable parameters. Instead of storing values for each (s, a) pair, the model compresses this information into a lower-dimensional parameter space $|\Theta| \leq |\mathcal{S} \times \mathcal{A}|$, enabling generalization across states (Plaats, 2022). As a result, DQN makes learning feasible in high-dimensional environments.

Why tabular update rules cannot be used directly In tabular Q-learning, updates are applied directly to individual entries using the Bellman optimality equation:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left(r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right).$$

This is possible because each $Q(s, a)$ is stored independently.

In contrast, DQN produces $Q_\theta(s, a)$ via a neural network with shared parameters θ . Updating θ to improve one state-action estimate affects many others. Therefore, individual values cannot be updated independently. Instead, we define a loss function and optimize it using gradient-based methods such as backpropagation.

Loss function in DQN The loss function in DQN is derived from the Bellman equation. For a transition (s, a, r, s') , the target is:

$$y = r + \gamma \max_{a'} Q_\theta(s', a').$$

The prediction is $Q_\theta(s, a)$, and the temporal-difference (TD) error is:

$$\delta = y - Q_\theta(s, a).$$

This leads to the mean squared error (MSE) loss:

$$\mathcal{L}(\theta) = \mathbb{E} \left[\left(r + \gamma \max_{a'} Q_\theta(s', a') - Q_\theta(s, a) \right)^2 \right].$$

Unlike supervised learning, the target depends on the network itself, making the loss a moving target (Plaats, 2022).

Instability and the Deadly Triad DQN combines three elements that can cause instability: function approximation, off-policy learning, and bootstrapping. Function approximation introduces interference due to shared parameters,

Algorithm 1 Deep Q-Learning with Experience Replay and Target Network

```

1: Initialize Q-network  $Q_{\theta_0}$  with random weights
2: Initialize target network  $Q_{\theta_0}$ 
3: Initialize replay buffer  $\mathcal{D}$ 
4: Set iteration counter  $i \leftarrow 0$ 
5: for each episode do
6:   Initialize state  $s$ 
7:   while episode not terminated do
8:     Select action  $a$  using policy  $\pi(s)$  derived from  $Q_{\theta_i}$ 
9:     Execute  $a$ , observe reward  $r$  and next state  $s'$ 
10:    Store transition  $(s, a, r, s')$  in  $\mathcal{D}$ 
11:    if  $\text{length}(\mathcal{D}) \geq \text{min\_length}$  then
12:      Sample mini-batch  $\{(s_j, a_j, r_j, s'_j)\}$  from  $\mathcal{D}$ 
13:      for each sampled transition do
14:         $y_j \leftarrow r_j + \gamma \max_{a'} Q_{\theta_{i-1}}(s'_j, a')$ 
15:      end for
16:       $\mathcal{L}(\theta_i) \leftarrow \frac{1}{N} \sum_j (y_j - Q_{\theta_i}(s_j, a_j))^2$ 
17:       $\theta_{i+1} \leftarrow \theta_i - \alpha \nabla \mathcal{L}(\theta_i)$ 
18:       $i \leftarrow i + 1$ 
19:      Every  $C$  steps:  $\theta_{i-1} \leftarrow \theta_i$ 
20:    end if
21:     $s \leftarrow s'$ 
22:  end while
23: end for
    
```

off-policy learning arises from evaluating a greedy policy while following an exploratory one, and bootstrapping relies on learned estimates. Their interaction, known as the “deadly triad”, can lead to divergence or oscillations.

Stabilization techniques: Experience Replay and Target Network To mitigate instability, two techniques can be applied. Experience Replay stores transitions $d = (s, a, r, s')$ in a buffer $\mathcal{D}$ and samples mini-batches uniformly at random. This breaks temporal correlations and improves sample efficiency by reusing past data.

The Target Network uses a separate network with parameters θ_{i-1} to compute targets:

$$y = r + \gamma \max_{a'} Q_{\theta_{i-1}}(s', a').$$

These parameters are periodically updated from the online network, reducing target non-stationarity.

DQN Algorithm The full DQN procedure with Experience Replay and a Target Network is summarized in Algorithm 1. The method alternates between collecting transitions and updating the Q-network by minimizing the temporal-difference loss over sampled mini-batches.

3. Experiments

In this section, we evaluate Deep Q-Learning (DQN) on the CartPole environment (Barto et al., 1983; Towers et al., 2024). We first describe the environment and the experimental setup shared across all experiments. We then conduct two studies: (i) an ablation study on the hyperparameters of a naive DQN implementation, and (ii) a comparison of DQN variants with and without Experience Replay (ER) and Target Networks (TN).

All experiments are conducted over 5 independent runs with different random seeds, each consisting of 10^6 environment steps. Every 5000 steps, the agent is evaluated over 10 episodes using a greedy policy (i.e., without exploration).

Data collection is vectorized over multiple environments, allowing several transitions to be generated in parallel at each step.

In the absence of a replay buffer, these parallel transitions are aggregated into a single batch and used directly for updates, resulting in a rollout-based training scheme. When using a replay buffer, transitions from all environments are stored and later sampled uniformly to construct mini-batches.

Environment We consider the CartPole environment (Barto et al., 1983; Towers et al., 2024), a classic control task introduced by Barto, Sutton, and Anderson. The system consists of a cart moving horizontally in $[-4.8, 4.8]$ and a pole rotating in $[-24^\circ, 24^\circ]$, both with unbounded velocities, where 0° denotes the upright position.

The state is defined by four continuous variables: cart position and velocity, and pole angle and angular velocity. The action space is discrete with two actions: moving the cart left or right.

The objective is to keep the pole balanced as long as possible. Episodes terminate if the cart leaves $[-2.4, 2.4]$, the pole exceeds $[-12^\circ, 12^\circ]$, or after 500 steps. Each timestep yields a reward of +1, resulting in a maximum return of 500.

Naive DQN: Hyperparameter Ablation We first analyze a naive DQN implementation without Experience Replay or a Target Network to assess the effect of key hyperparameters.

We vary the following hyperparameters independently:

- Exploration strategy: ϵ -greedy ($\epsilon \in \{0.01, 0.05, 0.2\}$) and softmax ($\tau \in \{0.5, 1, 2\}$)
- Network architecture: 1, 3, or 6 hidden layers
- Layer width: 64, 128, or 256 units

- Learning rate: $\{10^{-3}, 5 \cdot 10^{-4}, 10^{-4}, 10^{-5}\}$
- Data-to-update ratio: $\{1, 10, 32, 64\}$

Each configuration is evaluated over 5 runs, and results are averaged. The baseline uses a softmax policy with $\tau = 0.5$, learning rate $1 \cdot 10^{-3}$, data-to-update ratio 10, and a single hidden layer of 64 units.

Performance is measured using the Area Under the Curve (AUC) over the final 100,000 training steps. The best-performing setting in each group is selected based on this metric.

DQN Variants: ER and TN Using the best hyperparameter configuration, we evaluate four DQN variants: Naive DQN (baseline), DQN with Target Network (TN), DQN with Experience Replay (ER), and DQN with both TN and ER. The target network is updated every 101 training updates (rather than environment steps), ensuring that evaluation and target computation are not performed simultaneously. Experience replay uses a FIFO buffer of size 10^5 , with uniform sampling. Training starts after a minimum of 1000 transitions, introducing a short warm-up phase.

Results Table 1 reports the results of the hyperparameter study. The final configuration uses softmax exploration with $\tau = 1.0$, a single hidden layer of 64 units, a data-to-update ratio of 10, and a learning rate of $1 \cdot 10^{-3}$.

HP Group	Setting	AUC (10^7)
All	Baseline	3.98
Exploration	$\tau = 1.0$	4.62
Exploration	$\tau = 2.0$	4.42
Exploration	$\epsilon = 0.2$	4.07
Exploration	$\epsilon = 0.01$	3.79
Exploration	$\epsilon = 0.05$	3.34
Width	128 units	3.92
Width	256 units	3.80
Depth	6 layers	3.78
Depth	3 layers	3.26
Lr	$5 \cdot 10^{-4}$	3.82
Lr	$1 \cdot 10^{-4}$	0.89
Lr	$1 \cdot 10^{-5}$	0.09
Data-to-update	1	3.66
Data-to-update	32	2.62
Data-to-update	64	1.08

Table 1. AUC over the final 100,000 of training steps, averaged across five runs.

Figure 1 shows the corresponding learning curves. The results indicate that exploration strategy, learning rate, and data-to-update ratio have the strongest impact on performance, while network architecture (depth and width) has

only a limited effect. This suggests that, for this task, training dynamics are driven more by optimization and exploration than by model capacity.

Softmax exploration consistently outperforms ϵ -greedy, as it biases action selection toward higher-value actions while still allowing exploration. However, too low temperatures (e.g., $\tau = 0.5$) reduce exploration and degrade performance.

Although lower learning rates are generally associated with stability, the relatively small network used here allows for higher learning rates, leading to faster convergence and excessively large data-to-update ratios slow down learning and reduce performance.

Figure 2 compares the different DQN variants. The naive implementation achieves the best performance, indicating that for CartPole the basic formulation is already sufficient.

Both the target network and replay buffer introduce additional bias or delay in the learning signal. While these mechanisms are known to improve stability in more complex settings, in this task they primarily slow down convergence without clear benefits.

4. Discussion

The use of parallel environments in our implementation likely plays a central role in explaining the observed results. By collecting data from multiple environments simultaneously, transition correlations are significantly reduced, which is one of the primary motivations behind experience replay. As a result, the naive DQN implementation already benefits from a form of implicit de-correlation, likely explaining why it performs well despite the absence of explicit stabilization techniques such as a replay buffer or target network. In this context, it would be interesting to further investigate how the data-to-update ratio interacts with parallelism, for example by controlling this ratio within individual episodes, or test all the hyperparameter settings without parallelization.

Another limitation of this study is that it focuses solely on the main effects of hyperparameters. While the results indicate that relatively large learning rates are effective for smaller network architectures, interaction effects between hyperparameters were not examined. In particular, the relationship between network size and learning rate, as well as other potential interactions, could provide deeper insight into the behavior of the algorithm and should be explored in future work.

Furthermore, the replay buffer was implemented in such a way that it mirrors the data-to-update ratio of the naive agent. However, one of its key advantages is the ability to increase this ratio by reusing past experiences. Doing so may improve sample efficiency, but it also introduces addi-

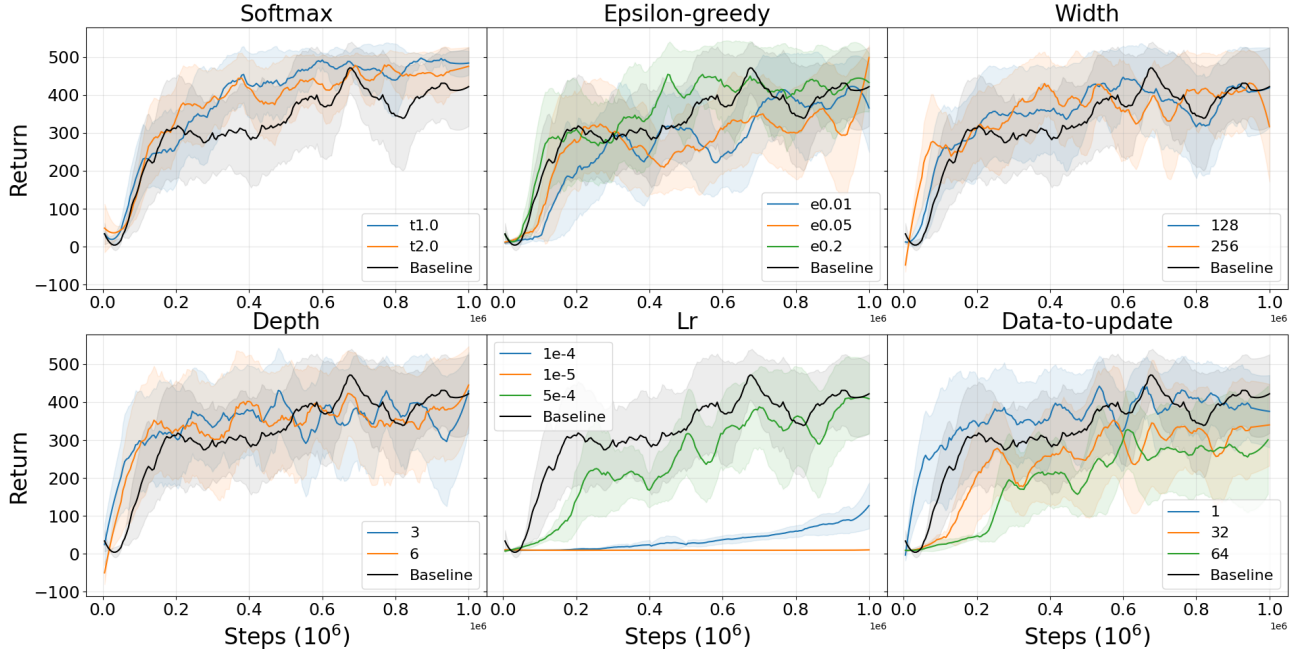


Figure 1. Learning curves for DQN under different hyperparameter settings. The black line represents the baseline configuration described in Sec. 3. The results indicate that the exploration strategy (and its associated parameters), learning rate, and data-to-update ratio have a significant impact on the agent’s performance. In contrast, the network architecture appears to have only a limited effect. A Savitzky–Golay filter, with a window size of 20, and a polynomial degree of 2 is used to smoothen the curves.

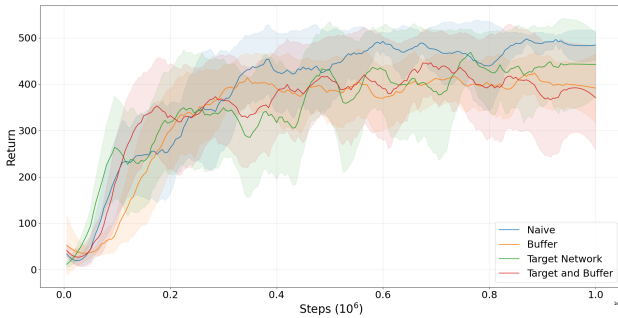


Figure 2. Learning curves for DQN variants: naive, TN, ER, and TN+ER. Results are averaged over multiple runs, with shaded regions indicating variance. A Savitzky–Golay filter, with a window size of 20, and a polynomial degree of 2 is used to smoothen the curves.

tional bias toward older transitions. Exploring this trade-off more systematically would provide a better understanding of when and how replay buffers become beneficial.

Finally, it remains unclear how well the identified hyperparameter configurations generalize to more challenging environments. Under increased task complexity, different design choices—such as larger networks or the explicit use of replay buffers and target networks—may become necessary. This is particularly relevant in light of the “deadly triad” of function approximation, off-policy learning, and

bootstrapping, which is known to cause instability in deep Q-learning. While such instability was not observed in this study, likely due to the implicit stabilization introduced by parallelization, a valuable direction for future work would be to characterize the conditions under which the naive approach breaks down and explicit stabilization techniques become essential.

5. Conclusion

We investigated Deep Q-Learning (DQN) on the CartPole environment through a combination of theoretical analysis and empirical evaluation. The hyperparameter ablation study revealed that learning rate, exploration strategy, and the data-to-update ratio have the most significant impact on performance, whereas network depth and width play a comparatively minor role. This indicates that, in this setting, learning dynamics are primarily governed by optimization and exploration rather than model capacity.

Using the best-performing configuration, we compared a naive DQN implementation with variants incorporating target networks and experience replay. Contrary to expectations, these stabilization techniques did not improve performance and in some cases slowed convergence. A likely explanation is that the use of parallel environments already reduces transition correlations, providing a form of implicit stabilization that diminishes the benefits of replay buffers

and target networks.

These findings suggest that the effectiveness of common DQN extensions is highly dependent on the problem setting and implementation details. While such techniques are essential in more complex and unstable environments, they may introduce unnecessary overhead in simpler domains.

Future work should investigate how these results generalize to more challenging tasks and explore the interaction between parallelization, replay mechanisms, and hyperparameters. In particular, identifying the conditions under which naive DQN becomes unstable would provide valuable insight into when explicit stabilization techniques are truly required.

References

- Barto, A. G., Sutton, R. S., and Anderson, C. W. Neuronlike adaptive elements that can solve difficult learning control problems. *IEEE Transactions on Systems, Man, and Cybernetics*, SMC-13(5):834–846, 1983. doi: 10.1109/TSMC.1983.6313077.
- Mnih, V. Playing atari with deep reinforcement learning. *arXiv preprint arXiv:1312.5602*, 2013.
- Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540): 529–533, 2015.
- Plaat, A. Deep reinforcement learning: A textbook. *arXiv preprint arXiv:2201.02135*, 2022. doi: 10.48550/arXiv.2201.02135.
- Towers, M. et al. Gymnasium: A standard interface for reinforcement learning environments. https://gymnasium.farama.org/environments/classic_control/cart_pole/, 2024. Accessed: 2026-04-01.

A. You *can* have an appendix here.

You can have as much text here as you want. The main body must be at most 8 pages long. For the final version, one more page can be added. If you want, you can use an appendix like this one.

The `\onecolumn` command above can be kept in place if you prefer a one-column appendix, or can be removed if you prefer a two-column appendix. Apart from this possible change, the style (font size, spacing, margins, page numbering, etc.) should be kept the same as the main body.